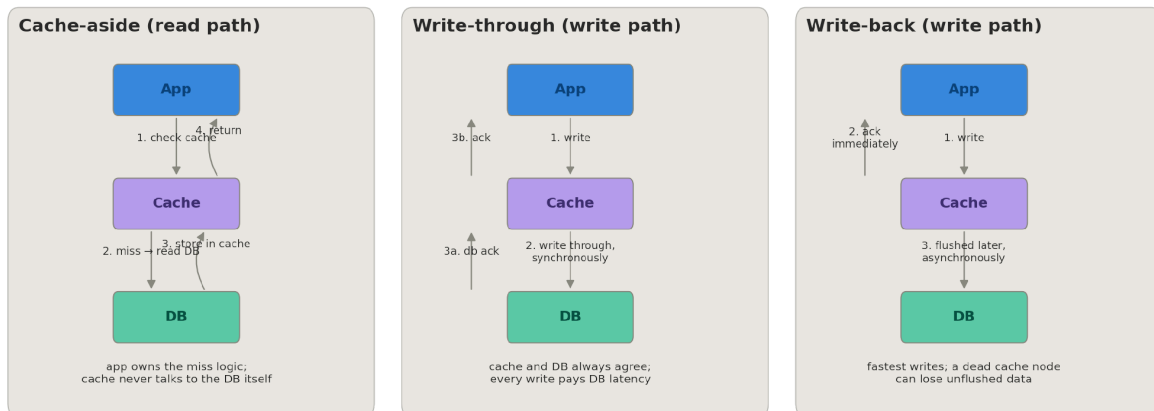


Stage 4 Crash Course: Caching

Caching only makes sense in front of something slow — and “slow” was just defined in Stage 3 (a DB round trip). Everything here assumes that source-of-truth store already exists.

1. Cache Patterns

These describe who talks to the cache, in what order, relative to the real data store.



- Cache-aside (lazy loading): the application checks the cache first; on a miss, it reads the DB itself and populates the cache. The cache is “dumb” — it never talks to the DB. Most common pattern in practice because it's simple and the cache only ever holds what's actually been requested.
- Read-through: functionally similar to cache-aside, but the cache library itself handles the miss — fetching from the DB transparently so the application code just asks the cache and never sees a miss path.
- Write-through: every write goes to the cache, which synchronously writes to the DB before acknowledging. Cache and DB never disagree, but every write pays full DB latency — you get correctness, not write speed.
- Write-back (write-behind): the cache acknowledges the write immediately and flushes to the DB later, asynchronously. Fastest writes, but if the cache node dies before flushing, that data is gone — a real durability risk that has to be explicitly accepted or mitigated (e.g., replicated cache).

Decision framing: cache-aside is the default for read-heavy workloads. Write-through earns its cost when correctness between cache and DB is non-negotiable. Write-back earns its cost only when write throughput matters more than the risk of losing the last few unflushed writes.

2. Eviction Policies

A cache has finite memory. When it's full and a new item needs to go in, something has to leave — the eviction policy decides what.



- LRU (Least Recently Used): evicts whatever hasn't been accessed in the longest time. Assumes recently-used data is likely to be used again soon — true for most real access patterns.
- LFU (Least Frequently Used): evicts whatever has been accessed the fewest total times. Better than LRU when “popular” and “recent” diverge — e.g., a genuinely popular item that just hasn't been hit in the last few minutes shouldn't be evicted the way LRU would evict it.
- TTL (Time To Live): every entry expires after a fixed duration regardless of usage. Not really a competing strategy — it's usually layered on top of LRU/LFU as a correctness bound, so nothing sits stale forever even if it stays popular.

Why this matters: the eviction policy is a bet about future access patterns based on past behavior. Picking wrong doesn't break correctness (cache-aside just re-fetches on a miss) — it shows up as a worse hit rate, i.e., more traffic silently landing back on your database than you'd expect.

3. Cache Invalidation

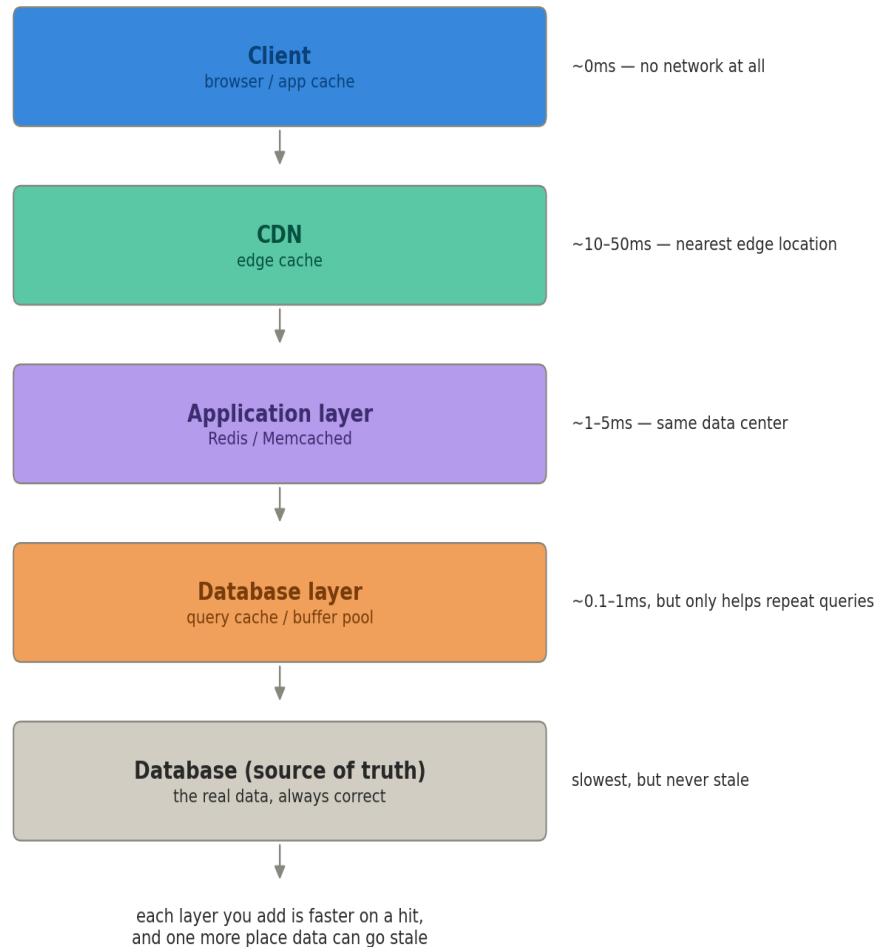
“There are only two hard things in computer science: cache invalidation and naming things.” The problem: the cache is a copy, and copies can go stale the moment the source of truth changes.

- Invalidate immediately: on every write, actively delete or update the cached copy. Correctness-first, but adds write-path complexity and coordination — especially across multiple cache nodes or regions.
- Short TTL: don't bother invalidating; just accept that data can be stale for up to the TTL window. Simple, but is a direct consistency-for-simplicity trade — the same PACELC tradeoff from Stage 1, now living inside your cache layer.

Thundering herd: when a hot cache key expires, many concurrent requests can all miss at once and hammer the DB simultaneously trying to refill it. Common mitigations: jittered TTLs (so keys don't all expire at the same instant) and request coalescing (only one request actually goes to the DB; the rest wait on that result).

4. Where to Cache

Caching isn't one layer — it exists at multiple points between the client and the source of truth, and real systems typically stack several of these.



Multi-layer coherence problem: more layers means more places the same piece of data can be stale relative to each other at the same time — a CDN might serve a version the app-layer cache has already invalidated. There isn't a clean fix for this beyond choosing, deliberately, how stale each layer is allowed to be and being honest about it in the design, rather than assuming all caches agree.

How this stage connects back

- Caching only exists because Stage 3's storage layer has latency (Stage 1's latency numbers) worth avoiding on repeat reads.
- Cache invalidation is the same consistency-vs-latency tradeoff as CAP/PACELC (Stage 1) and replication lag (Stage 3), just at a layer you control more directly.
- Eviction and invalidation are different problems: eviction is about making room when the cache is full; invalidation is about correctness when the source of truth changes. Confusing the two is a common interview mistake.

Must be able to explain before moving on

1. Why write-back is the fastest write pattern and what real risk that speed introduces.
2. The difference between LRU and LFU, and a concrete access pattern where LFU would give a better hit rate.
3. Why thundering herd happens, and one concrete mitigation for it.
4. Why adding more cache layers (client → CDN → app → DB) doesn't just add speed — explain what it costs in return.